Informe Técnico y Auditoría de Desarrollo: Juego del Ahorcado Asistido por IA

1. Contexto del Proyecto y Ficha Técnica

En el marco de la educación superior técnica, la adopción de herramientas de Inteligencia Artificial Generativa (IAG) para la ingeniería de software exige un análisis riguroso que trascienda la mera medición cuantitativa de la velocidad de codificación. La realización de auditorías técnicas sobre desarrollos asistidos por LLMs resulta indispensable para evaluar la convergencia entre la aceleración en la entrega de artefactos digitales y la asimilación conceptual de la arquitectura por parte del alumno. Este informe evalúa la construcción del proyecto "Juego del Ahorcado Web", auditando la metodología de interacción mediante prompts, la validez lógica del código generado, las estrategias de refactorización aplicadas y el grado real de soberanía técnica alcanzado frente a las soluciones sugeridas por la IA.

Ficha Técnica del Proyecto

Parámetro	Detalle Institucional y Técnico
Nombre del Proyecto	Juego del Ahorcado Web
Asignatura	Desarrollo Asistido con IA (DAI)
Curso y Titulación	2º G.D. C.F.G.S. D.A.W.

Centro Educativo	IES Trafalgar

Arquitectura del Juego

El sistema presenta una arquitectura cliente pura, fundamentada en una separación de responsabilidades a través de tres módulos esenciales: `index.html` para la estructura semántica y accesible, `styles.css` para la capa de presentación fluida y reactiva, y `script.js` para la orquestación del estado del juego y los eventos del DOM. Un pilar central de esta arquitectura es el motor de sonido desacoplado (`SoundEngine`), sustentado en la API nativa `AudioContext` de la plataforma web. Esta aproximación sintetiza efectos sonoros y secuencias melódicas retro en tiempo real mediante osciladores programáticos, eliminando la latencia y las dependencias de assets multimedia externos (archivos MP3/WAV). Asimismo, la representación gráfica del ahorcado se articula mediante renderizado vectorial SVG dinámico, cuyos nodos de trazo son controlados e inyectados incrementalmente por el controlador JavaScript en respuesta a la mutación del contador de errores.

La distribución cuantitativa de la base de código que se analiza a continuación permite determinar el alcance de la intervención humana frente a la generación automatizada en la consecución de esta arquitectura.

2. Auditoría y Desglose de Contribución del Código

El análisis forense de la procedencia del código es un requisito clave para auditar el software asistido por IA. Categorizar la base de código según su origen —manual, generado directamente por el LLM o refactorizado a posteriori— permite medir la autonomía técnica real del desarrollador. Este desglose evidencia si la herramienta de IA funcionó como un simple generador monolítico de código o como un catalizador integrado dentro de un ciclo riguroso de ingeniería, supervisión y optimización.

Distribución Porcentual de Autoría

Categoría	Porcentaje	Descripción del Trabajo y Componentes Afines
Propietario / Manual	**10%**	Maquetación manual de la estructura de archivos local, supervisión del entorno en IDE, pruebas de usabilidad e interfaz, y parametrización de constantes de diseño y tiempos.
Generación Directa por IA	**55%**	Estructura de maquetación HTML base, banco léxico inicial, controladores de eventos del DOM y lógica nuclear de síntesis de onda en el motor de audio.
Refactorización / Optimización	**35%**	Ajuste fino del tempo musical de Tetris, reestructuración del layout a una sola línea para la palabra oculta, maquetación CSS Responsive adaptativa (móviles/tablets) y sincronización de revelación completa del estado al perder.

Evaluación del Impacto de la IA

La distribución de autoría refleja un desplazamiento fundamental en el rol del desarrollador: se pasa de la codificación sintáctica artesanal a la orquestación y auditoría de componentes. Que más de un tercio del código final (35%) corresponda a tareas de refactorización directa demuestra que el código emitido por la IA, aunque funcional en sus bloques elementales, requiere una intervención correctiva para cumplir con estándares de experiencia de usuario (UX), diseño adaptativo y robustez de estado.

Esta estructura de autoría es el resultado de un proceso iterativo de diálogo técnico documentado formalmente en la bitácora de interacción.

3. Bitácora Completa de Interacción e Ingeniería de Prompts

La trazabilidad del proceso de desarrollo asistido reside en la documentación metódica de los prompts. Esta bitácora cronológica no solo registra las peticiones formales realizadas al LLM (Gemini), sino que expone la evolución del producto desde un prototipo funcional rudimentario hasta una aplicación web refinada y adaptativa.

Registro Cronológico de Interacciones

Para conciliar la traza documental con la metodología de desarrollo, se registran las 9 iteraciones explícitas de prompting enviadas al modelo, culminando en la fase imprescindible de validación técnica, integración y despliegue final en el entorno de desarrollo local (Paso 10):

1. **Prompt Inicial (Arquitectura y Motor Base):** Definición de la estructura cliente del juego con estética moderna, segmentación modular (`index.html`, `styles.css`, `script.js`) y diseño del motor `SoundEngine` basado en `Web Audio API` para sintetizar eventos sonoros (inicio, aciertos, fallos, victoria, derrota).
2. **Ajuste Estético Interno:** Inserción de la melodía de fondo de Tetris, diferenciación cromática del contenedor interior respecto al fondo exterior para mejorar el contraste visual y escalado del contenedor principal.
3. **Interactividad Visual y Control de Estado:** Implementación de un teclado virtual dinámico para la entrada por puntero, deshabilitado de teclas consumidas para evitar estados redundantes, integración de isotipo en el encabezado, animaciones CSS de entrada para caracteres y representación vectorial SVG evolutiva del ahorcado según el recuento de errores.
4. **Variabilidad Sonora:** Ampliación de la secuencia de notas de la melodía de Tetris en el sintetizador para prevenir la monotonía acústica en sesiones prolongadas.

5. **Simetría en el Fin de Juego:** Reestructuración de la condición de derrota para equiparar la interfaz modal de fin de partida a la de victoria, forzando la revelación explícita de la palabra oculta.
6. **Layout Visual y Flujo Horizontal:** Modificación de las reglas de maquetación para restringir los contenedores de caracteres a una sola línea horizontal continua sin saltos de línea.
7. **Calidad y Selección del Léxico:** Sustitución del banco de palabras arcaicas o infrecuentes devuelto por el LLM por un array de términos en castellano de uso común pero con alta complejidad de resolución.
8. **Ajuste del Tempo Sonora:** Consulta técnica para aislar y modificar los parámetros temporales de la música de fondo, ralentizando el ritmo base mediante la constante `speedFactor`.
9. **Diseño Adaptativo (Responsive CSS):** Reestructuración integral de las hojas de estilo mediante reglas relativas y *media queries* para garantizar la adaptabilidad en dispositivos móviles, tablets y monitores de alta densidad.
10. **Integración, Validación y Despliegue (Fase Final):** Ciclo de auditoría manual en IDE (Visual Studio Code), verificación de la integridad del DOM, pruebas cross-browser del motor `AudioContext` y empaquetado final de la aplicación.

Muestra de Prompts Representativos y Análisis Auditable

Caso A: Arquitectura Inicial y Motor de Audio Native

Prompt de entrada: *"Diseña una página web similar a la práctica del ahorcado. Quiero que el diseño sea moderno con colores atractivos, con teclado virtual en pantalla, animación en las letras, dibujo dinámico en SVG del ahorcado y música de fondo estilo Tetris con sonidos adaptativos mediante Web Audio API."*

Estructura devuelta:

- Desacople en tres archivos independientes (`index.html`, `styles.css`, `script.js`).

- Clase `SoundEngine` que abstrae `AudioContext` para sintetizar frecuencias sin assets externos.

Evaluación técnica: La respuesta inicial ofreció una separación de responsabilidades arquitectónicamente sólida. La abstracción sonora mediante `AudioContext` eliminó dependencias I/O de red, garantizando un arranque instantáneo.

Caso B: Resolución de Simetría y Gestión de Estado en Defección

Prompt de entrada: *"Quiero que si se pierde muestre un mensaje igual que cuando acierta la palabra y que muestre la palabra completa que había que adivinar."*

Solución aplicada e inspección de código: Modificación del manejador de eventos `handleLetterGuess()` en `script.js`. Al alcanzar el límite estipulado de 7 fallos, el controlador ejecuta:

guessedWord = currentWord.split('');

Auditoría de Arquitectura de Estado: Desde la perspectiva del control de estado, la asignación `guessedWord = currentWord.split('')` fuerza una mutación determinista sobre el modelo de datos local antes de la emisión del evento de renderizado UI y la posterior llamada al diálogo modal. Esto evita condiciones de carrera donde la interfaz podría congelarse mostrando únicamente las letras acertadas hasta ese momento. La refactorización garantiza la sincronización total entre la variable de estado interna y la representación visual en el DOM en transiciones límite (*edge cases*).

La evaluación de este proceso de interacción revela la necesidad de analizar los casos más complejos de depuración lógica y optimización de código.

4. Casos de Estudio de Debugging, Detección de Alucinaciones y Refactorización

La auditoría técnica identificó inconsistencias en las respuestas del LLM, desde sesgos en el cálculo temporal de la API de Audio hasta problemas de contención de desbordamiento en CSS. Esta sección documenta la intervención de ingeniería aplicada para depurar estos fallos.

Caso 1: Sincronización Temporal de Audio y Control del Tempo (Debugging/Alucinación)

- **Problema:** Se detectó una aceleración anómala y descontrolada en la cadencia de la música de fondo conforme avanzaba la partida. La IA incurrió en una alucinación lógica al vincular el avance de las notas de la melodía al contador de eventos/fallos del juego en lugar de utilizar un reloj de tiempo real independiente. Esto provocaba que cometer errores acelerase drásticamente la tasa de reproducción, arruinando la cadencia acústica.
- **Solución Aplicada:** Se aisló el bucle de reproducción del ciclo de eventos del DOM, implementando el reloj de alta precisión `performance.now()` para calcular intervalos de tiempo real en milisegundos (`MIN_TRACK_DURATION_MS`). Para controlar la velocidad, se parametrizó la constante de retardo mediante la fórmula:

const speedFactor = Math.max(550, 800 - (currentErrors * 30));

Nota de auditoría temporal: En esta ecuación, un valor alto en milisegundos representa un intervalo de espera mayor entre notas, es decir, un tempo **más lento**. Al aumentar `currentErrors`, el valor de `speedFactor` disminuye gradualmente hasta el tope de `550 ms`, acelerando la música de forma sutil y controlada para incrementar la tensión dramática de la partida, eliminando la aceleración caótica inicial.

- **Impacto:** Desacople absoluto entre el estado del bucle del juego y el hilo de síntesis sonora, asegurando una cadencia armónica predecible.

Caso 2: Adaptabilidad Visual Responsiva y Control de Desbordamiento (Refactorización CSS)

- **Estrategia de Maquetación Fluida:** Para evitar la rotura del contenedor en pantallas estrechas, se sustituyeron los anchos fijos en píxeles por la función de maquetación CSS `clamp()`. Un ejemplo representativo de esta refactorización es la tipografía de las casillas:

font-size: clamp(1rem, 2.5vw, 1.8rem);

Esta sintaxis combina una unidad fluida basada en el viewport (`2.5vw`) con un límite inferior de seguridad (`1rem`) y un techo máximo de renderizado (`1.8rem`), garantizando que la fuente se escale de forma continua sin invadir márgenes adyacentes.

- **Técnicas de Flexbox y Media Queries:** Para impedir que la palabra a adivinar colapsara en dos líneas en pantallas móviles pequeñas, se aplicaron las propiedades `flex-wrap: nowrap` y `overflow-x: auto` sobre el contenedor `.word-display`. Complementariamente, se establecieron puntos de ruptura tácticos mediante `@media (max-width: 850px)` y `@media (max-width: 480px)` para ajustar el *padding* y los márgenes de los botones del teclado virtual.

Tabla Comparativa de Desafíos Técnicos

Caso Auditoría / Problema Detectado	**Causa en el Código de la IA**	**Solución Refactorizada**	**Impacto en la UX**

Sincronización Sonora	Vinculación del avance de notas al recuento de fallos/eventos en lugar de un reloj de tiempo real.	Uso de `performance.now()` y control temporal inverso mediante `speedFactor`.	Transición musical armónica, fluida y desacoplada del ritmo de pulsación del usuario.
Layout Responsivo	Contenedores rígidos con anchos fijos y desbordamiento de línea en palabras largas.	Implementación de `clamp()`, `flex-wrap: nowrap` y *Media Queries* específicas.	Visualización continua y sin saltos de línea de la palabra en cualquier dispositivo.
Simetría de Victoria/Derrota	Desincronización entre el estado interno de la palabra y el DOM al agotar los 7 intentos.	Mutación explícita `guessedWord = currentWord.split('')` previa al renderizado modal.	Cierre cognitivo completo y satisfacción de la expectativa del usuario al perder.

El diagnóstico de estos casos técnicos permite fundamentar la autoevaluación sobre las competencias adquiridas durante el desarrollo.

5. Autoevaluación Crítica y Reflexión sobre el Rol de Arquitecto de Software

Productividad vs. Comprensión Técnica

La utilización de la Inteligencia Artificial Generativa derivó en un incremento estimado del **50%** en la velocidad global de entrega,

permitiendo integrar en pocas horas un motor de síntesis de audio retro que, mediante desarrollo tradicional sin asistentes, habría requerido jornadas completas de investigación sobre especificaciones W3C de `Web Audio API`.

Desde la perspectiva de la defensa del código, el desarrollador demuestra una capacidad de justificación técnica superior al **90%** en componentes críticos: manipulación sintáctica del DOM, arquitectura de eventos, maquetación basada en CSS Grid/Flexbox y la lógica de la máquina de estados del juego.

Sin embargo, la auditoría pone de manifiesto el riesgo del **"ingreso de código tipo caja negra"** (*black-box code ingestion*). En el módulo `script.js`, la construcción del sintetizador abarca la instanciación de nodos osciladores (`OscillatorNode`) y controles de ganancia (`GainNode`), manipulando directamente curvas de envolvente acústica y frecuencias matemáticas en hercios (Hz). Aunque el desarrollador comprende el flujo lógico general, la formulación matemática pura de las ondas sonoras generadas por la IA representa una capa de complejidad asimilada sin un dominio matemático profundo de base, lo que supone un punto ciego que requiere especial supervisión en entornos de producción profesional.

Fricciones Técnicas y Puntos Ciegos

- **Puntos de Fricción:** La mayor dificultad técnica radicó en ajustar el comportamiento dinámico dentro de la función `playTetrisMusic()`. Fue necesario realizar múltiples iteraciones para ajustar los retardos y evitar la superposición de ondas (*clipping*) al modificar la velocidad del tempo.
- **Puntos Ciegos en Desarrollo Autónomo:** Se identifica como principal limitación ante un eventual desarrollo *offline* la imposibilidad de memorizar la tabla de frecuencias armónicas (Hz) necesarias para programar notas musicales desde cero mediante `AudioContext` sin la asistencia de un modelo de lenguaje.

Evolución Profesional: El Desarrollador como Arquitecto y Auditor

La ejecución de este proyecto consolida un cambio de paradigma en el perfil profesional del desarrollador de software:

- **De la sintaxis manual a la especificación técnica:** La competencia clave deja de ser el tecleo de código repetitivo para centrarse en la descomposición de problemas complejos en especificaciones y prompts estructurados.
- **Auditoría crítica y detección de alucinaciones:** La labor principal exige validar de forma continua los artefactos generados por los LLMs, corrigiendo inconsistencias lógicas, modelos de datos desincronizados y dependencias temporales erróneas.
- **Refactorización orientada a la experiencia de usuario:** La intervención humana garantiza que el código abstracto producido por la IA se adapte a estándares reales de usabilidad, accesibilidad y diseño responsivo.

La auditoría técnica concluye validando la viabilidad del modelo de desarrollo asistido por Inteligencia Artificial, confirmando que la aceleración en los tiempos de entrega mantiene su valor técnico siempre que se ejerza un control de calidad riguroso sobre el producto final.

/

MANUAL PARA LA CREACIÓN DEL JUEGO EL AHORCADO CON IA.

1/ Compartí con gemini IA el pdf y con el prompt donde le pedí
PRIMER PROMPT: " vamos a crear un juego donde te voy a pasar toda la información a través de un pdf, se trata del juego el ahorcado, quiero que el diseño sea moderno con colores atractivos y con una música adaptada al juego y que haga sonidos adaptativos para cuando vaya acertando letras, para cuando falles en letras, cuando inicie el juego, y cuando termine, un sonido de haber ganado y un sonido cuando pierda el jugador, genera el código en archivos diferentes según el código, html, css y javaScript **":**

2/ Abrí el VisualStudio Codec y aunque podía descargarme los archivos desde la web de la IA, creé a mano los archivos y pegué los tres códigos.

3/ Aunque el juego funciona bien y la música con los tonos, no me terminaba de gustar y le pedí que:
SEGUNDO PROMPT: quiero otro tipo de musica de fondo como la melodia del tetris, el fondo del interior del juego me gustaria que fuera de color distinto al exterior y algo mas claro, tambien un poco mas grande

4/ El juego ha mejorado pero aún necesita más cambios visuales y acústicos para mi gusto.
TERCER PROMPT: quiero que para las letras haya un teclado para pulsar con el ratón, también quisiera que se reflejaran las letras que vaya usando y que no se pudieran repetir, el tono de perdedor me gustaria como el que te voy a pasar por audio, tambien quisiera que el título de JUEGO DEL AHORCADO tuviera un logo, y cada vez que comience una partida tuviera las letras movimientos dinámicos simpaticos, tambien quisiera un dibujo dinámico con movimientos de un muñeco del ahorcado moderno, que tuviera efectos visuales según se vaya avanzando con el juego

5/ El juego me encanta, lo único que voy a cambiar es que la música sea más variada, porque cansa el mismo tono.
CUARTO PROMPT: añade la melodia completa del juego tetris.

6/ Probando el juego me di cuenta que cuando se perdía la partida no mostraba la palabra que había que adivinar:
QUINTO PROMPT: quiero que si se pierde muestre un mensaje igual que cuando acierta la palabra y que muestre la palabra completa que había que adivinar.

7/ La palabra a adivinar siempre aparecía en dos líneas y no me gustaba, le pedí que modificara el código para que la palabra estuviera en una sola línea.
SEXTO PROMPT: quiero que en el juego la palabra a adivinar esté en una sola línea, cambia el ancho del juego**.**

8/ También me dí probando jugando con el juego que las palabras eran poco comunes y algo extrañas para mi, así que le pedí un array con palabras más comunes.
SÉPTIMO PROMPT: las palabras son muy raras, haz un array con palabras más comunes aunque sean difíciles de adivinar.

9/ Me parecía que la música iba muy rápida y le pedí:
OCTAVO PROMPT: indica que parte del código debo cambiar manualmente para que la música vaya más lenta.
// Música más lenta (Aumentamos los milisegundos base)
const speedFactor = Math.max(550, 800 - (currentErrors * 30));

10/ NOVENO PROMPT: Quiero que me modifiques el ccs para que sea RESPONSIVE y pueda verse en pantallas de móviles y tablets, además de portátiles y monitores.

***/**

NOTA: Le pedí a gemini Notebook pasandole el pdf del proyecto terminado y le pedí que le diera un toque profesional para la presentación en clase.

```
# DOCUMENTO DE PROCESO Y AUDITORÍA DE IA

Nombre del Proyecto: Juego del Ahorcado Web
Asignatura: Desarrollo Asistido con IA (DAI)
Curso: 2° G.D. C.F.G.S. D.A.W.
Centro: IES Trafalgar

## 1. Registro de Herramientas y Desglose de Contribución

### Herramientas Utilizadas
- Gemini (Modelo LLM Assistant): Utilizado para la generación de la estructura HTML5, diseño y maquetación CSS3 responsiva, lógica de juego en JavaScript, y la implementación del motor de síntesis de audio con "Web Audio API".

### Estimación Porcentual del Código Final
- % Código / diseño propio: 10% (Maquetación manual, supervisión, prueba de usabilidad y parametrización de constantes de diseño y tiempos).
- % Código / diseño directo de IA: 55% (Estructura de maquetación, banco de palabras, eventos del DOM y lógica del sintetizador de audio).
- % Código / diseño refactorizado: 35% (Ajuste de tiempos de la música de Tetris, conversión del diseño a una sola línea para la palabra, implementación de Responsive CSS en móviles/tablets y lógica de revelación completa al perder).

## 2. Bitácora de Interacción (Muestra de Prompts Reales)

### Generación / Arquitectura
```

- Prompt de entrada enviado:
  "Diseña una página web similar a la práctica del ahorcado. Quiero que el diseño sea moderno con colores atractivos, con teclado virtual en pantalla, animación en las letras, dibujo dinámico en SVG del ahorcado y música de fondo estilo Tetris con sonidos adaptativos mediante Web Audio API."
- Código/Estructura devuelta:
  - Estructura modular dividida en `index.html`, `styles.css` y `script.js`.
  - Clase `SoundEngine` basada en `AudioContext` para sintetizar notas musicales sin necesidad de archivos MP3/WAV externos.
- Análisis: La arquitectura inicial funcionó perfectamente. Permitió separar responsabilidades y construir una interfaz fluida sin dependencias externas.

Resolución de errores / Debugging

- Prompt de entrada enviado:
  > *"Quiero que si se pierde muestre un mensaje igual que cuando acierta la palabra y que muestre la palabra completa que había que adivinar."*
- Solución propuesta:
  - Modificación de la función `handleLetterGuess()` en `script.js` para que al llegar a 7 fallos se asigne `guessedWord = currentWord.split('')` antes de actualizar la UI y lanzar la ventana modal emergente.
- Evaluación: Solucionó el problema de inmediato, logrando una experiencia de usuario totalmente simétrica tanto al ganar como al perder.

Detección de alucinaciones / Código erróneo

- Prompt de entrada enviado:
  "Quiero que la música de fondo del tetris sea completa."
- Caso detectado: La IA intentó cambiar de canciones basándose en el número de notas procesadas en lugar del tiempo real transcurrido, lo que hacía que en palabras con fallos la música fuese demasiado rápido.

- *xplicación del fallo y corrección: Se detectó que el tiempo musical dependía de la velocidad del bucle. Se corrigió forzando el uso de `performance.now()` para medir intervalos exactos de tiempo real en milisegundos (`MIN_TRACK_DURATION_MS`).

Refactorización / Optimización

- Prompt de entrada enviado:
  > *"Quiero que me modifiques el CSS para que se pueda ver en pantallas de móviles y tablets además de portátiles y monitores, y que la palabra no haga saltos de línea."*
- **Código/CSS adaptado**:
  - Incorporación de `clamp()` para tipografías y tamaños de casillas adaptativos.
  - Inserción de `flex-wrap: nowrap` y `overflow-x: auto` en `.word-display`.
  - Definición de Media Queries (`@media (max-width: 850px)` y `@media (max-width: 480px)`).

3. Autoevaluación Crítica y Reflexión Personal

1. Productividad vs. Comprensión

La Inteligencia Artificial ha incrementado la velocidad de desarrollo en un **50%**, permitiendo implementar funcionalidades avanzadas como la síntesis de audio retro en tiempo récord. Sería capaz de explicar y defender **más del 90%** del código subido al repositorio, especialmente la manipulación del DOM, los manejadores de eventos, las reglas CSS Grid/Flexbox y la lógica del estado del juego. El codigo del archivo script.js, no.

2. Fricciones y Bloqueos

Los principales momentos de fricción ocurrieron al ajustar el comportamiento dinámico del motor de audio (tiempos de reproducción y ralentización del tempo), donde fue necesario iterar varias veces sobre la función `playTetrisMusic()` hasta dar con el ritmo exacto deseado.

3. Puntos Ciegos

En caso de tener que modificar el proyecto en vivo sin conexión ni asistentes, la principal debilidad radicaría en el cálculo matemático preciso de las frecuencias en hercios (Hz) para la síntesis de notas musicales en Web Audio API de memoria.

4. Impacto en el Flujo de Trabajo

El flujo de trabajo ha evolucionado de escribir sintaxis línea a línea a asumir un rol de ****arquitecto y auditor de código****. Ahora el enfoque principal reside en saber descomponer el problema en requisitos técnicos claros, evaluar la calidad de las soluciones propuestas por la IA y refactorizar el código para garantizar la máxima usabilidad y accesibilidad.